Marevlo Research / AI Foundations • Module 05 – Batching • Topic 3

Lossless Rejection Sampling, Worked Out by Hand

Why speculative decoding has *zero* quality loss — a complete proof

What this document does. Topic 2 showed speculative decoding emits several tokens per target pass. That would be worthless if it changed the model's output distribution. This document *proves* it does not. The modified rejection-sampling rule — accept the draft's token x with probability $\min(1, p(x)/q(x))$, and on rejection resample from the normalized residual $\max(0, p-q)$ — emits tokens with *exactly* the target distribution p, for *any* draft q. The whole proof collapses to one identity: $\min(p, q) + \max(0, p - q) = p$. We prove that identity, then verify the full result on a tiny 4-token vocabulary, by hand and by Monte-Carlo. Reproduced by `m05_t03_rejection_sampling_engine.py` (Appendix A).

The one idea. Speculative decoding is not an approximation. The acceptance probability was *engineered* so that the two ways of emitting a token — accept it from the draft, or resample it after a rejection — always sum to exactly $p(x)$.

Concepts covered, in order
the sampling procedure; the two disjoint emission paths; Path A mass = $\min(p, q)$; Path B mass = $\max(0, p-q)$; the core identity $\min(p, q) + \max(0, p-q) = p$ (proved); a 4-token worked verification (✓ per token); and a Monte-Carlo confirmation.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

Contents

1 The procedure

Let p be the target (big-model) distribution and q the draft (small-model) distribution over vocabulary V. One emitted token is produced as follows.

Modified rejection sampling — the distributional identity

1. Draft samples $x \sim q(\cdot)$.
2. With probability $\min\big(1,\, p(x)/q(x)\big)$, **accept** and emit x.
3. Otherwise **reject** x and resample from the normalized residual

$$r(y) \;=\; \frac{\max\big(0,\, p(y) - q(y)\big)}{\sum_z \max\big(0,\, p(z) - q(z)\big)}.$$

Claim: the emitted token's distribution is *exactly* p.

Intuition

The draft over-proposes some tokens and under-proposes others. Where $q(x) > p(x)$ (the draft is too eager), acceptance $\min(1, p/q) < 1$ throws some of those proposals back. Where $q(x) < p(x)$ (the draft is too shy), the residual $\max(0, p-q)$ adds exactly the missing mass back on rejection. The acceptance rule and the residual are two halves of one bookkeeping system that always lands on p.

2 Step 1 — The core identity

Everything rests on one elementary fact about real numbers.

$\min(a,b) + \max(0, a-b) = a$ for any $a, b \geq 0$

Two cases, by sign of $a - b$:

$$\begin{aligned}
&\textbf{Case } a \geq b: && \min(a,b) = b, \quad \max(0, a-b) = a-b, \\
&&& \text{sum } = b + (a-b) = a. \;\checkmark \\
&\textbf{Case } a < b: && \min(a,b) = a, \quad \max(0, a-b) = 0, \\
&&& \text{sum } = a + 0 = a. \;\checkmark
\end{aligned}$$

In both cases the sum is a. Setting $a = p(x)$, $b = q(x)$:

$$\boxed{\min\big(p(x), q(x)\big) + \max\big(0,\, p(x) - q(x)\big) = p(x).}$$

The entire lossless guarantee is this identity. Steps 2–4 show that the procedure's two emission paths contribute exactly the two terms on the left.

3 Step 2 — The two disjoint paths to emitting x

A fixed token x can be emitted in exactly two mutually exclusive ways.

Path A — draft proposed x and we accepted it.

$$\begin{aligned}\Pr(\mathrm{A}, x) &= \underbrace{q(x)}_{\text{draft picks } x} \times \underbrace{\min\big(1,\, p(x)/q(x)\big)}_{\text{accept}} \\ &= \min\big(q(x),\, p(x)\big) \quad \text{(distribute } q(x) \text{ inside the min)}.\end{aligned}$$

Path B — the draft proposed some z, we rejected, then resampled x. First, the total rejection probability:

$$\Pr(\text{reject}) = \sum_z q(z)\Big(1 - \min\big(1, p(z)/q(z)\big)\Big) = \sum_z \max\big(0,\, q(z) - p(z)\big).$$

This equals the residual normalizer $Z = \sum_z \max(0, p(z) - q(z))$, because the total "excess of q over p" equals the total "deficit of q below p" (both distributions sum to 1). Then

$$\Pr(\mathrm{B}, x) = \Pr(\text{reject}) \times r(x) = Z \times \frac{\max(0, p(x) - q(x))}{Z} = \max\big(0,\, p(x) - q(x)\big).$$

The two terms are exactly the identity's two terms

$\Pr(\mathrm{A}, x) = \min(p(x), q(x))$ and $\Pr(\mathrm{B}, x) = \max(0, p(x) - q(x))$. The normalizer Z cancels: the rejection mass is precisely the mass the residual needs to redistribute. Adding the two paths invokes the Step 1 identity.

4 Step 3 — Assembling the proof

$\Pr(x) = p(x)$

The two paths are disjoint and exhaustive, so

$$\Pr(x) = \Pr(\mathrm{A}, x) + \Pr(\mathrm{B}, x) = \underbrace{\min\big(p(x), q(x)\big)}_{\text{Path A}} + \underbrace{\max\big(0,\, p(x) - q(x)\big)}_{\text{Path B}} = p(x),$$

by the identity of Step 1. This holds for *every* $x \in V$ and for *any* draft q with $\mathrm{supp}(p) \subseteq \mathrm{supp}(q)$. ■

Watch out

The one precondition: $q(x) > 0$ wherever $p(x) > 0$ (the draft must give nonzero probability to anything the target might emit), so $p(x)/q(x)$ is well-defined. In practice the draft shares the target's vocabulary and uses a softmax, so every token has positive mass and the condition holds automatically. A draft that hard-zeros tokens the target can produce would violate losslessness.

5 Step 4 — A worked verification on a 4-token vocabulary

Let $V = \{A, B, C, D\}$ with a target p the draft q approximates badly:

	A	B	C	D
p	0.50	0.30	0.15	0.05
q	0.20	0.40	0.30	0.10

The draft over-weights B, C, D and badly under-weights A. We verify the procedure still lands on p.

5.1 Acceptance and Path-A mass

$\Pr(\mathrm{A}, x) = q(x) \cdot \min(1, p(x)/q(x)) = \min(p(x), q(x))$:

	A	B	C	D
$p(x)/q(x)$	2.50	0.75	0.50	0.50
$\min(1, p/q)$	1.00	0.75	0.50	0.50
$\Pr(\mathrm{A}, x) = \min(p, q)$	0.20	0.30	0.15	0.05

Total accept mass $\sum \min(p, q) = 0.20 + 0.30 + 0.15 + 0.05 = 0.70$, so $\Pr(\text{reject}) = 0.30$.

5.2 The residual distribution

$Z = \sum_y \max(0, p(y) - q(y))$. Only A has $p > q$:

$$\max(0, p - q): \quad A{:}\,0.30, \quad B{:}\,0, \quad C{:}\,0, \quad D{:}\,0 \quad \Rightarrow \quad Z = 0.30 = \Pr(\text{reject}). \checkmark$$

So the residual is $r(A) = 0.30/0.30 = 1$, $r(B) = r(C) = r(D) = 0$: every rejection resamples A, exactly the token the draft starved.

5.3 Total emission probability

$\Pr(x) = \Pr(\mathrm{A}, x) + \Pr(\text{reject}) \cdot r(x)$:

x	Path A	Path B $= 0.30 \cdot r(x)$	$\Pr(x)$	$p(x)$	
A	0.20	0.30	0.50	0.50	✓
B	0.30	0.00	0.30	0.30	✓
C	0.15	0.00	0.15	0.15	✓
D	0.05	0.00	0.05	0.05	✓

Every row matches p exactly. The badly-mismatched draft did not bias the output at all — it only changed *how often* we hit the cheap accept path vs. the corrective resample path. ✓

Mini-example — reading the result

Token A shows the mechanism cleanly. The draft proposes A only 20% of the time, but the target wants it 50%. The accept path delivers 0.20; the missing 0.30 is restored entirely through rejections, every one of which resamples A. Accept mass plus residual mass $= 0.20 + 0.30 = 0.50 = p(A)$. The identity, made arithmetic.

6 Step 5 — Monte-Carlo confirmation

Running the literal procedure 2,000,000 times (draft-propose, accept-or-resample) gives empirical frequencies indistinguishable from p:

x	A	B	C	D
empirical	0.4998	0.2999	0.1503	0.0499
$p(x)$	0.5000	0.3000	0.1500	0.0500

The deviations are pure Monte-Carlo noise ($O(1/\sqrt{N}) \approx 7 \times 10^{-4}$). The distribution is p, not "approximately p." ✓

Lossless means lossless

Because the proof collapses to the algebraic identity $\min(p, q) + \max(0, p - q) = p$, there is no quality knob, no accuracy tradeoff, no "mostly as good." Speculative decoding produces *the target model's own distribution*, sampled through a cheaper procedure. When someone reports quality loss, the cause is an implementation bug — most often a draft/target *temperature mismatch* that makes the q in the code differ from the q used to sample — not the algorithm.

7 The argument, at a glance

#	Step	Result
1	Identity	$\min(p, q) + \max(0, p - q) = p$ (two cases)
2	Two paths	$\Pr(\text{A}) = \min(p, q)$; $\Pr(\text{B}) = \max(0, p - q)$
3	Proof	$\Pr(x) = \min(p, q) + \max(0, p - q) = p(x)$ ■
4	Worked	4-token vocab: $\Pr(x) = p(x)$ for all x ✓
5	Monte-Carlo	2M draws match p to noise ✓

7.1 Equation cheat-sheet

Acceptance prob.	$a(x) = \min\big(1,\ p(x)/q(x)\big)$
Path A mass	$q(x)\, a(x) = \min\big(p(x), q(x)\big)$
Rejection prob.	$\sum_z \max(0, q(z) - p(z)) = Z$
Residual	$r(y) = \max(0, p(y) - q(y)) \,/\, Z$
Path B mass	$Z \cdot r(x) = \max\big(0,\ p(x) - q(x)\big)$
Core identity	$\min(p, q) + \max(0, p - q) = p$
Result	$\Pr(x) = p(x) \quad \forall x$

Appendix A — Reference implementation

Every number above is produced by `m05_t03_rejection_sampling_engine.py`. It tabulates the two paths, checks $\Pr(x) = p(x)$ exactly, and runs the 2-million-draw Monte-Carlo confirmation.

```
"""
Reference implementation for: Module 05 - Topic 3
"Modified Rejection Sampling - why speculative decoding is LOSSLESS."

```

```
We verify, on a tiny 4-token vocabulary, that the modified rejection-sampling
procedure emits tokens with EXACTLY the target distribution p, regardless of
the draft distribution q. The identity that makes this work is

    min(p, q) + max(0, p - q) = p   (for any non-negative p, q).

Every number printed here is transcribed into the worked-by-hand PDF.
"""

# ---------------------------------------------------------------------
# 0. A tiny vocabulary: V = {A, B, C, D}.
#    p = target (big model) distribution; q = draft (small model) distribution.
# ---------------------------------------------------------------------
V = ["A", "B", "C", "D"]
p = {"A": 0.50, "B": 0.30, "C": 0.15, "D": 0.05}   # target
q = {"A": 0.20, "B": 0.40, "C": 0.30, "D": 0.10}   # draft

print("== 0. The two distributions ==")
print(f"{'tok':>4} {'p(x)':>7} {'q(x)':>7}")
for x in V:
    print(f"{x:>4} {p[x]:>7.3f} {q[x]:>7.3f}")
assert abs(sum(p.values()) - 1) < 1e-12 and abs(sum(q.values()) - 1) < 1e-12

# ---------------------------------------------------------------------
# 1. Acceptance probability per token: a(x) = min(1, p(x)/q(x)).
#    Path-A mass (draft sampled x AND accepted) = q(x)*a(x) = min(p,q).
# ---------------------------------------------------------------------
print("\n== 1. Acceptance and Path-A mass ==")
print(f"{'tok':>4} {'p/q':>7} {'a=min(1,p/q)':>14} {'q*a=min(p,q)':>14}")
accept_mass = {}
for x in V:
    a = min(1.0, p[x] / q[x])
    accept_mass[x] = q[x] * a              # = min(p[x], q[x])
    print(f"{x:>4} {p[x]/q[x]:>7.3f} {a:>14.4f} {accept_mass[x]:>14.4f}")

# ---------------------------------------------------------------------
# 2. Overall rejection probability and the residual distribution.
#    Pr(reject) = 1 - sum_x min(p,q) = sum_x max(0, q-p)
#    residual r(y) = max(0, p(y)-q(y)) / Z,  Z = sum_y max(0, p-q)
# ---------------------------------------------------------------------
P_accept = sum(accept_mass.values())       # sum min(p,q)
P_reject = 1.0 - P_accept
Z = sum(max(0.0, p[y] - q[y]) for y in V)   # normalizer of residual
print("\n== 2. Rejection mass and residual ==")
print(f"Pr(accept) = sum min(p,q) = {P_accept:.4f}")
print(f"Pr(reject) = 1 - Pr(accept) = {P_reject:.4f}")
print(f"Z = sum max(0,p-q) = {Z:.4f}   (equals Pr(reject): {abs(Z-P_reject)<1e-12})")
print(f"{'tok':>4} {'max(0,p-q)':>12} {'r(y)=./Z':>10}")
resid = {}
for y in V:
    pos = max(0.0, p[y] - q[y])
    resid[y] = pos / Z if Z > 0 else 0.0
    print(f"{y:>4} {pos:>12.4f} {resid[y]:>10.4f}")

# ---------------------------------------------------------------------
# 3. Total emission probability per token and the identity check.
#    Pr(x) = Path A + Path B
#          = min(p,q) + Pr(reject)*r(x)
#          = min(p,q) + Z * max(0,p-q)/Z
#          = min(p,q) + max(0,p-q)
#          = p(x).
# ---------------------------------------------------------------------
print("\n== 3. Total emission probability  Pr(x) =?= p(x) ==")
```

```
print(f"{'tok':>4} {'pathA':>8} {'pathB':>8} {'Pr(x)':>8} {'p(x)':>8} {'ok':>4}")
all_ok = True
for x in V:
    pathA = accept_mass[x]                       # min(p,q)
    pathB = P_reject * resid[x]                   # Pr(reject)*r(x) = max(0,p-q)
    total = pathA + pathB
    ok = abs(total - p[x]) < 1e-12
    all_ok = all_ok and ok
    print(f"{x:>4} {pathA:>8.4f} {pathB:>8.4f} {total:>8.4f} {p[x]:>8.4f} "
          f"{'OK' if ok else 'XX':>4}")
print(f"\nALL tokens match target p exactly: {all_ok}")

# ---------------------------------------------------------------------
# 4. Monte-Carlo sanity check (the procedure, run as code).
# ---------------------------------------------------------------------
import random
random.seed(0)
def sample_from(dist):
    u, c = random.random(), 0.0
    for k, v in dist.items():
        c += v
        if u <= c:
            return k
    return list(dist)[-1]

N = 2_000_000
counts = {x: 0 for x in V}
for _ in range(N):
    x = sample_from(q)                            # draft proposes
    if random.random() <= min(1.0, p[x] / q[x]):  # accept?
        counts[x] += 1
    else:
        counts[sample_from(resid)] += 1           # resample from residual
print("\n== 4. Monte-Carlo over %d draws ==" % N)
print(f"{'tok':>4} {'empirical':>10} {'p(x)':>8}")
for x in V:
    print(f"{x:>4} {counts[x]/N:>10.4f} {p[x]:>8.4f}")
```

— END OF DOCUMENT —